Transformers for Solving ARC-AGI 1: A Comparative Study with Data Augmentation

Eduardo Altmann, Thiago Noll e Vítor Feijó

Universidade Federal do Rio Grande do Sul— Instituto de Informática

Abstract

The Abstraction and Reasoning Corpus (ARC-AGI) is a challenging benchmark designed to evaluate few-shot abstract reasoning and generalization. In this study, we evaluate an independently trained vision Transformer architecture (`mini-arc-v12`, 67.3M parameters) across three controlled test-time configurations on 114 benchmark tasks of ARC-AGI: (1) a baseline with standard test-time training (TTT) under canonical orientation, (2) an invariance-driven data augmentation approach, and (3) an empirical upper-bound configuration enriched with procedurally generated verified demonstrations. Our findings demonstrate that strong data augmentation more than doubles exact-match task resolution from 4.39% to 9.65%, while elevating cell accuracy to 91.40%. Furthermore, expanding context density via verified demonstrations reaches 14.91% post-refinement (and 17.54% pre-refinement).

Introduction

ARC-AGI (Abstraction and Reasoning Corpus) was proposed by (Chollet 2019) as a benchmark for evaluating machine intelligence through skill acquisition and generalization of a few-shot reasoning scenario. Recent approaches to ARC include small Transformer models (Fletcher-Hill 2024), test-time training, perspective-based augmentation, and hybrid induction-transduction systems (Akyürek et al. 2025; Franzen, Disselhoff, and Hartmann 2024; Li et al. 2024). Each ARC task consists of input-output pairs of colored grids that demonstrate a transformation rule, and the solver must infer and apply this rule to a new input.

In this work, we investigate an independently trained vision Transformer architecture derived from Mini-ARC (Fletcher-Hill 2024) for solving ARC-AGI-1 tasks. Rather than evaluating disparate neural architectures, we hold the underlying solver fixed and conduct a strictly controlled, paired comparison across three test-time configurations on 114 benchmark tasks:

1. **Baseline:** Standard test-time training (TTT) utilizing only the original task demonstrations under the canonical orientation;
2. **Data Augmentation:** An invariance-driven strategy expanding the TTT adaptation pool across all eight D_4 geometric symmetries, seeded color permutations, and demonstration orderings, followed by multi-candidate canonical inversion and hierarchical consensus voting at inference;
3. **Privileged Upper Bound (Cheat):** The baseline procedure augmented with compatible procedurally generated demonstrations from ARC-GEN (Moffitt 2025). This scenario serves as an empirical positive control to measure the value of dense task-specific supervision, isolating whether the model is primarily bottlenecked by representation ambiguity or by demonstration sample size.

Data augmentation (Akyürek et al. 2025; Franzen, Disselhoff, and Hartmann 2024; Li et al. 2024) is a widely used technique for increasing the diversity of available inputs. In visual tasks, transformations such as rotation and reflection can generate alternative views that preserve the underlying rule that can improve generalization from few examples.

The third configuration studies a different source of additional information. Extra input-output pairs are provided, and the Transformer uses these pairs as additional demonstrations. Unlike geometric augmentation, this procedure does not merely change the presentation of an existing pair: it increases the number of examples available to the solver.

Comparing these configurations isolates two possible benefits of extra inference-time information: invariance to alternative views and access to a larger set of examples. The study provides a basis for analyzing how the amount and source of additional demonstrations influence accuracy and generalization on ARC-AGI tasks under a shared model checkpoint.

Hypotheses

We hypothesize that the baseline configuration will achieve the lowest performance because it provides the solver with the least inference-time information. Geometric and color data augmentation is expected to produce a substantial improvement by exposing the solver to equivalent views of the same demonstrations and encouraging spatial consistency. Furthermore, the privileged generated-example configuration is expected to yield the highest performance because it supplies verified additional demonstrations, directly addressing the few-shot ambiguity bottleneck. Thus, our expected performance ordering is the baseline, followed by data augmentation, followed by the privileged upper bound.

Related Work

ARC-AGI and Few-Shot Abstract Reasoning

The Abstraction and Reasoning Corpus (ARC) was introduced by Chollet (Chollet 2019) as a benchmark for evaluating skill acquisition and generalization rather than performance obtained through extensive task-specific training. Each task contains only a small number of input-output grid pairs, and the solver must infer a transformation rule and apply it to a new input. Consequently, success depends not only on recognizing visual patterns, but also on selecting a rule that generalizes beyond the observed examples.

Transformers for ARC

Transformers use self-attention to model relationships between elements in a sequence (Vaswani et al. 2023). This mechanism is attractive for ARC because a grid transformation may depend on relationships between distant cells or objects. However, a Transformer receives only a few demonstrations for each task, which makes it difficult to learn a new transformation reliably without task-specific adaptation.

Fletcher-Hill (Fletcher-Hill 2024) investigates this setting with small Transformer models trained on ARC puzzles. The Mini-ARC system combines a 67-million-parameter Transformer with test-time training and a refinement procedure, showing that a relatively small model can solve a meaningful subset of ARC tasks.

Data Augmentation and Multiple Perspectives

In conventional machine learning, data augmentation increases the diversity of training data while preserving the target concept. In ARC, geometric transformations can serve a related but more specific purpose: a rotation, reflection, or transposition changes the presentation of a task while preserving the relation between its input and output. Predictions from transformed views can then be mapped back to the original orientation and compared.

Franzen et al. (Franzen, Disselhoff, and Hartmann 2024) use this perspective explicitly in The LLM ARChitect. Their method generates and evaluates candidate solutions across multiple geometric views, using consistency across those views to select promising answers. Augmentation is therefore used not only to create more examples, but also to reduce dependence on a single representation of the same task. Akyürek et al. (Akyürek et al. 2025) similarly apply transformations during both task adaptation and inference, although their approach also changes model parameters through TTT.

Procedural Benchmark Generation

A related line of work targets the ARC-AGI dataset itself, procedurally expanding the pool of verified $\langle \text{input}, \text{output} \rangle$ pairs available for each task rather than transforming or synthesizing examples at inference time. Moffitt (Moffitt 2025) introduces ARC-GEN, an open-source generator designed to cover all original ARC-AGI-1 training tasks while closely preserving the distributional properties of the official dataset, in contrast to earlier procedural generators that offer only partial task coverage or looser fidelity to the original examples.

Methodology

ARC-AGI Dataset and RE-ARC Preparation

The Abstraction and Reasoning Corpus (ARC-AGI) benchmark evaluates general intelligence through visual grid transformation tasks. For pre-training our model, we utilize a reduced 12×12 subset of the synthetic RE-ARC dataset (Hodel 2024). To prevent data imbalance, the raw task files undergo a filtering process, tasks with grid dimensions different to 12×12 or generator families containing fewer than six total examples are discarded (Fletcher-Hill 2024). The resulting dataset consists of 391 eligible generator families and 186,556 total retained examples (Fletcher-Hill 2024).

Transformer Model Architecture

Our primary neural architecture, `mini-arc-v12`, is a spatial patch-based vision encoder derived from the ARC Vision Encoder design (Fletcher-Hill 2024). The architecture maps flat sequence inputs derived from grid image patches into spatial predictions (Fletcher-Hill 2024).

Grid Tokenization and Input Sequence. Each 12×12 grid is spatially discretized into 2×2 non-overlapping patches (Fletcher-Hill 2024). Input tasks are formatted as a sequence consisting of four demonstration input-output grid pairs $(X_i, Y_i)_{i=1}^{4}$ followed by a single test query grid X_{test} (Fletcher-Hill 2024). Each patch embedding combines patch color representations with learned spatial and positional embeddings.

Architecture Configurations. We implement three distinct profile configurations to evaluate scale and target refinement dynamics:

- **Baseline Profile:** Replicates the full paper specification containing 16 encoder layers, 16 attention heads, $d_{\text{model}} = 512$, and $d_{\text{ff}} = 3072$ ($\approx 67.3 \times 10^6$ parameters) (Fletcher-Hill 2024).
- **Full-Refinement Profile:** Uses the same 67.3M parameter architecture but is pre-trained with noisy partial targets passed through an auxiliary `tgt_embedding` input path on 25% of optimization steps (Fletcher-Hill 2024). On the remaining 75% of steps, it predicts directly from learned output queries (Fletcher-Hill 2024). This pre-training regime conditions the model to perform multi-pass iterative refinement on its own predictions during test-time inference (Fletcher-Hill 2024).

Loss Function. The model is trained end-to-end using cross-entropy loss computed over the predicted cell color classes across all spatial positions of the target output grid (Fletcher-Hill 2024).

Data Augmentation

Our augmentation approach expands test-time training (TTT) and ensembling into an identity-anchored sampling strategy operating on D_4 symmetries, seeded color permutations, and demonstration orderings. It avoids non-deterministic generation while retaining strict canonical mapping for voting.

For a given task, TTT examples are sampled up to a fixed budget of 256 items from the Cartesian product of

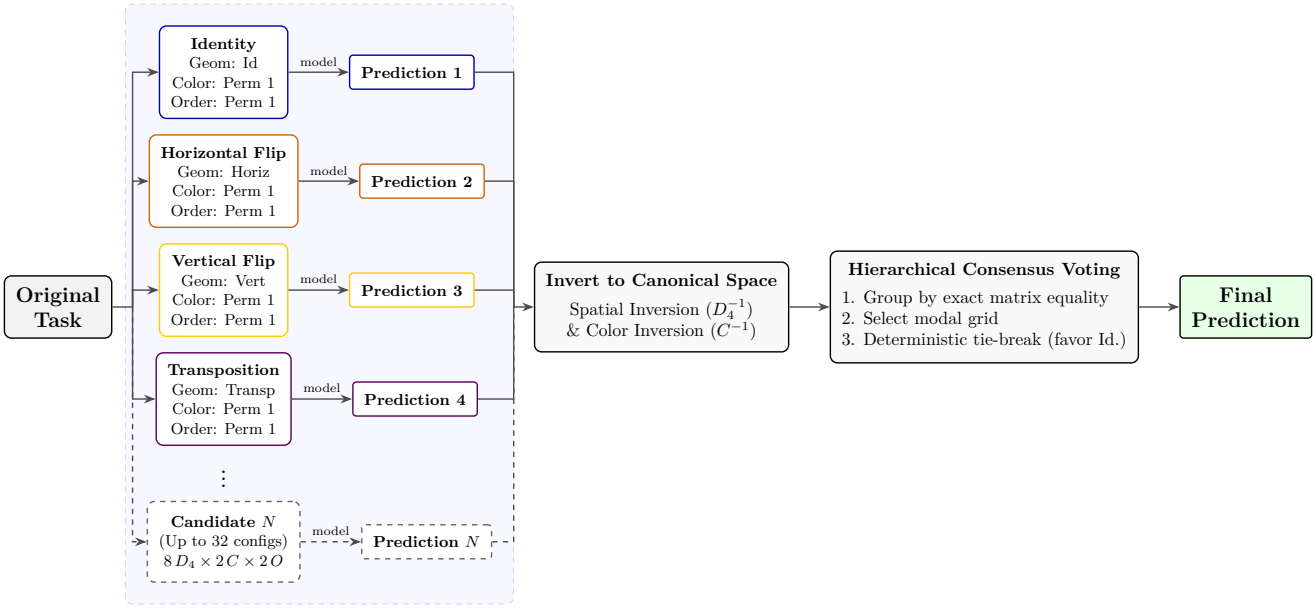

Figure 1: Overview of the data augmentation and ensembling process via identity-anchored sampling and hierarchical consensus voting.

all eight D_4 geometric symmetries, four seeded color permutations (preserving the background color 0), and demonstration order permutations. Across the 114 evaluation tasks, this procedure derives 24,512 training items from 94,912 candidates before capping (averaging 215 items per task). A controlled identity fraction (identity_fraction $= 0.25$) ensures that canonical views remain represented in the pool. At inference time, the ensemble generates predictions across up to 32 candidate configurations (comprising eight geometric views, two color mappings, and two demonstration orderings).

Each output candidate is inverted back to the canonical task space via its corresponding spatial and color inverse mappings. Final predictions are selected using hierarchical consensus voting: candidate grids are grouped by exact matrix equality, selecting the modal grid, with ties resolved deterministically in favor of predictions supported by the canonical identity view.

Generated Demonstrations (via Google API)

Our example-generation method, which we refer to as exemplar synthesis, uses a stronger model as a controlled oracle: it is shown the labeled examples of a task, asked to infer the underlying transformation rule, and then used to synthesize new input-output pairs consistent with that rule. It is a zero-shot, inference-time augmentation technique that requires no weight updates or gradient-based training. The model used for generation is never the model used to solve the task, since a solver capable of inferring the rule precisely enough to produce valid synthetic data would, by definition, already be able to answer the task directly.

Context Formulation. For each task, the prompt includes every available training pair, together with the test inputs with their outputs removed, so that the generating model does not receive the correct answer during synthesis. Using the complete set of demonstrations, rather than an arbitrary subset, avoids discarding examples that may be informative. If C denotes the training context and X_{test} the unlabeled test inputs, the model receives

$$(C, X_{\text{test}}) \longrightarrow (r, f, G, \hat{Y}_{\text{test}}),$$

where r is a short natural-language explanation of the inferred rule, f is a Python function implementing that rule as `transform(grid)`, G is the set of synthesized training pairs, and $\hat{Y}_{\text{test}}$ are the predicted test outputs. The prompt further requires all generated grids to be rectangular matrices of integers and instructs the model to preserve height and width whenever these dimensions are constant across the original examples, preventing the synthesis process from altering the structure of the task without supporting evidence. Generation is performed in a single inference call per task. A single parameter, `generated_examples`, controls how many additional pairs are requested inside that one response, without multiplying the number of requests issued: for N tasks and no retries, the expected number of API calls is

$$R = N,$$

while the number of requested synthetic examples is approximately

$$E = N \times \texttt{generated\_examples}.$$

The number of retry attempts after transient errors (e.g., rate limiting or temporary service unavailability) is itself

a configurable parameter, t; when retries are enabled, the number of model calls is bounded by $R \leq N \times (1 + t)$. Since a failed call may still consume part of the request budget even when it produces no output file, the number of resulting augmented files is not a reliable proxy for the number of requests made.

Execution-Based Validation. Unlike a self-reported validity flag, the correctness of each pair is verified independently, in two stages. First, f is compiled and executed inside a restricted sandbox that disallows arbitrary Python syntax and denies access to imports, the filesystem, the network, or any external state; it is applied to every original pair, and if it fails to reproduce any original output exactly, all synthetic pairs for that task are discarded and only the original demonstrations are retained. Second, once the original pairs are reproduced correctly, each generated pair is checked against a set of structural invariants derived from the original examples: every grid must be a well-formed matrix (non-empty, rectangular, and integer-valued), and, whenever a property – height, width, the set of colors present, or the estimated background color – is constant across all valid original grids on a given side (input or output), the same value is required of the corresponding generated grids. If any generated pair violates one of these invariants, the entire batch of synthetic pairs for that task is discarded. Otherwise, f is applied individually to each generated pair, and its output is compared for exact equality against the expected grid; only pairs that pass this functional check are retained. Properties that are not constant across the original examples – such as color usage, object connectivity, or symmetry – are left unconstrained, since a non-constant property may itself be the key to the underlying rule. This execution-based check, rather than any claim made by the model, determines the semantic validity of each example; visual inspection of rendered grids serves as a complementary qualitative baseline for a subset of tasks, while the sandboxed execution provides scalable, deterministic verification for the full set.

Task Augmentation. When validation succeeds, an augmented task file is assembled as

$$C^+ = C \cup G,$$

together with the unlabeled test inputs and metadata identifying the source task and generation time; the predicted test outputs $\hat{Y}_{\text{test}}$ are kept only as an auxiliary signal and are not part of the augmented training set. The main computational cost of this method is a single call per task to a stronger model – `gemini-3.5-flash-lite` in the current configuration – with a high internal-reasoning setting enabled. Reproducibility was not enforced through a fixed random seed, since some variation across runs was treated as a useful source of diversity before a formal comparison protocol was established; determinism can be introduced later for controlled ablations.

Generation Statistics. Table 1 summarizes the outcome of this pipeline across the 114 evaluation tasks. Of the 1,090 synthetic pairs requested (10 per task), 54.5% were accepted, though acceptance was strongly bimodal at the task level: 41 tasks (37.6%) had all 10 candidates accepted, while 40 tasks (36.7%) had all candidates rejected – the latter driven largely by structural invariant violations, which discarded the entire synthetic batch for 37 tasks (33.9% of expanded tasks). The five failed tasks split across a persistent API outage (one task, despite five configured retry attempts), semantic validation failures (three tasks), and an unsupported syntax construct in a generated transformation function (one task); only transient API errors triggered automatic retries.

Table 1: Outcome of the Google API generation pipeline across 114 evaluation tasks.

Metric	**Value**
Tasks processed	114
Successfully expanded	109 (95.6%)
Failed (API / validation / syntax)	5 (4.4%)
Original examples (mean $\pm$ SD)	3.80 ± 1.39
Synthetic pairs requested	1,090
Synthetic pairs accepted	594 (54.5%)
Tasks fully accepted (10/10)	41 (37.6%)
Tasks fully rejected (0/10)	40 (36.7%)
Tasks partially accepted (1–9/10)	28 (25.7%)
Tasks discarded via invariant violation	37 (33.9%)

Generated Demonstrations (via ARC-GEN)

As an alternative source of verified demonstrations, we employ ARC-GEN (Moffitt 2025), an open-source procedural generator that provides a hand-authored, parameterized generation function for each of the four-hundred original ARC-AGI-1 training tasks, together with a `validate()` routine confirming that its outputs reproduce the task's original demonstrations exactly. Because each generator is constructed to match the precise transformation rule of its corresponding task, examples drawn from it are correct by construction, without requiring the execution-based sandboxing and invariant checking used in the Google API configuration. For each of the 108/114 evaluation tasks with a compatible generator, we sample 10 additional $\langle$input, output$\rangle$ pairs via `generate()` and add them to the training context, following the same augmentation rule $C^+ = C \cup G$ described previously.

Experimental Configuration and Evaluation Protocol

Infrastructure and Training Environment. Pretraining and fine-tuning are performed using Distributed Data Parallelism (DDP) across 8 $\times$ NVIDIA A100 GPUs on the Grid'5000 cluster environment of PCAD(http://pcad.inf.ufrgs.br). Code execution is containerized via Apptainer using a PyTorch 2.4.1 and CUDA 12.1 runtime environment.

Hyperparameters and Optimization. Model parameters are optimized using AdamW with learning rate $\eta = 10^{-4}$ and weight decay $\lambda = 10^{-4}$ under Automatic Mixed Precision

(AMP) (Fletcher-Hill 2024). Models are trained with a per-GPU batch size of 16 (yielding a global batch size of 128 across 8 GPUs) for up to 150,000 steps (≈ 150 epochs at 1,000 training steps per epoch). Model checkpoints are saved atomically based on the lowest validation loss achieved on the held-out RE-ARC validation split (Fletcher-Hill 2024).

Test-Time Training (TTT) Adaptation. During ARC-AGI evaluation, we apply Test-Time Training (TTT) to adapt the base model to individual tasks (Akyürek et al. 2025; Fletcher-Hill 2024):

1. For each ARC puzzle, a temporary copy of the pre-trained base model checkpoint is instantiated (Fletcher-Hill 2024).
2. The copied model is adapted on context permutations generated from all combinations containing at least three demonstration pairs (e.g., yielding 6 items for 3 pairs, or 48 items for 4 pairs) (Fletcher-Hill 2024).
3. Fine-tuning runs for up to 15 epochs per puzzle or terminates early upon reaching a 99.5% adaptation-accuracy threshold (Fletcher-Hill 2024).
4. For models trained under the `full-refinement` profile, two iterative inference refinement passes are performed through the `tgt_embedding` pass after TTT adaptation (Fletcher-Hill 2024).
5. The adapted model predicts the test query grid, after which its local state is discarded without modifying the base checkpoint (Fletcher-Hill 2024).

Evaluation Metrics. Evaluation results are reported using three standard metrics across the 114 test tasks (Fletcher-Hill 2024):

- **Score (Exact Match):** The percentage and count of evaluation tasks where the predicted query grid matches the ground-truth output matrix exactly across 100% of cells (Fletcher-Hill 2024).
- **Cell Accuracy:** The overall percentage of correctly predicted individual cells across all padded 12×12 grids (Fletcher-Hill 2024).
- **Closeness:** The percentage of tasks where the prediction achieves at least 95% cell accuracy relative to the target grid (Fletcher-Hill 2024).

Results

In this section, we present the empirical results of our evaluation across the 114 standardized ARC-AGI benchmark tasks (Fletcher-Hill 2024). We compare three strictly controlled test-time configurations on the identical `mini-arc-v12-full-refinement` checkpoint: the baseline Transformer solver, our strong data augmentation ensemble, and the privileged upper-bound configuration enriched with procedurally generated ARC-GEN demonstrations.

Comparative Benchmark Performance

Table 2 reports the primary evaluation metrics across the three configurations at the post-refinement stage (matching the standard TTT + Refined setting).

Table 2: Comparative performance on the 114 ARC-AGI evaluation tasks across inference configurations (post-refinement stage, matching the paper's TTT + Refined setting).

Configuration	Score	Score (%)	Cell Acc. (%)	Closeness (%)
Baseline	5 / 114	4.39%	89.80%	40 / 114 (35.09%)
Data Augmentation	**11 / 114**	**9.65%**	**91.40%**	**44 / 114 (38.60%)**
Privileged Upper Bound	17 / 114	14.91%	92.90%	62 / 114 (54.39%)

The baseline configuration, utilizing standard TTT with only original demonstrations under canonical orientation, solves 5 out of 114 tasks, achieving an exact match score of 4.39%, an overall cell accuracy of 89.80%, and reaching $\geq 95\%$ closeness on 40 tasks (35.09%). Note that while cell accuracy appears moderately high ($\approx 90\%$), this metric is inflated by the background padding required to embed grids into a uniform 12×12 matrix; exact-match Score remains the decisive metric for task resolution.

The Impact of Strong Data Augmentation and Context Density

Applying our symmetry-aware data augmentation strategy produced a substantial performance gain, more than doubling the exact-match Score from 5 to 11 tasks out of 114 (9.65%, a $2.2\times$ relative improvement). Concurrently, cell accuracy improved by $+1.60$ percentage points (from 89.80% to 91.40%) and the number of close predictions increased from 40 to 44 tasks.

A granular task-level transition analysis reveals that data augmentation is not uniformly additive. Augmentation successfully resolves eight tasks missed by the baseline (`5783df64`, `6ea4a07e`, `94414823`, `af24b4cc`, `b1fc8b8e`, `c48954c1`, `c7d4e6ad`, `ef26cbf6`) while preserving three tasks solved by the baseline (`68b67ca3`, `e633a9e5`, `fc754716`). However, it regresses on two tasks previously solved by the baseline (`3b4c2228`, `9110e3c5`), yielding a net gain of $+6$ tasks. This indicates that while multi-view sampling effectively breaks single-view inductive bias, ensembling across transformed views can occasionally dilute a valid canonical prediction when spatial symmetries conflict.

The privileged upper bound condition (augmenting the context with compatible ARC-GEN procedural pairs) resolved 17 tasks post-refinement (14.91%) and reached a closeness score of 54.39% ($62/114$ tasks), fixing 13 tasks relative to the baseline while regressing on only one (`3b4c2228`). Supplying verified additional demonstration pairs directly expands the target context during test-time fine-tuning, providing sufficient gradient signals to eliminate function ambiguity that cannot be resolved from the minimal original context alone.

Dynamics of Test-Time Refinement

In the original Mini-ARC framework (Fletcher-Hill 2024), two rounds of iterative refinement are treated as a universally positive post-processing step. As documented in Table 3, our experiments uncover a nuanced interaction between test-time supervision volume and refinement stability.

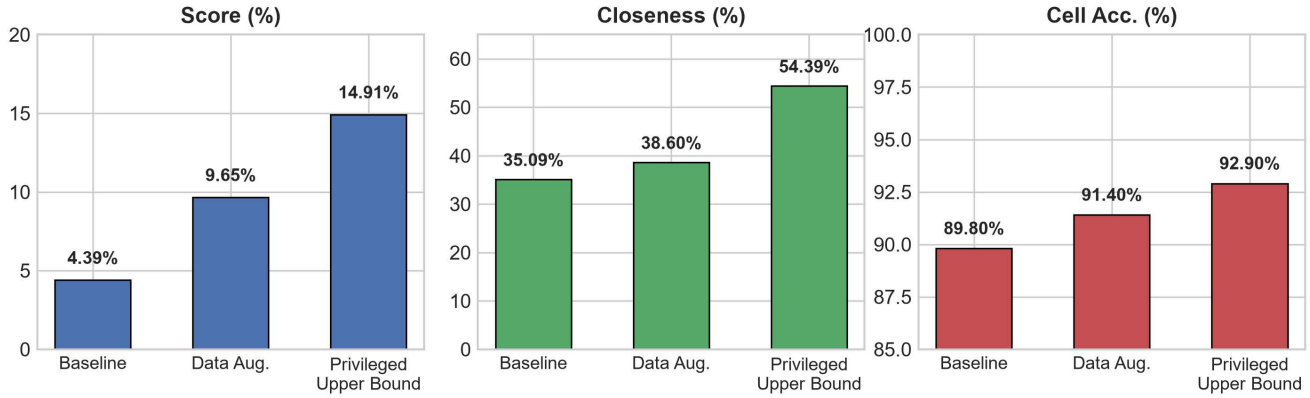

Figure 2: Overview of the data augmentation and ensembling process via identity-anchored sampling and hierarchical consensus voting.

Table 3: Ablation of test-time refinement (two inference correction passes) on exact-match Score across 114 tasks.

Configuration	TTT Score	TTT + Refined	Change
Baseline	5 / 114 (4.39%)	5 / 114 (4.39%)	0
Data Augmentation	9 / 114 (7.89%)	11 / 114 (9.65%)	+2
Privileged Upper Bound	20 / 114 (17.54%)	17 / 114 (14.91%)	-3

While refinement provides a net boost of $+2$ solved tasks for data augmentation (rising from 9 to 11 tasks) by polishing partially correct consensus predictions, it actively degraded performance in the dense privileged upper-bound condition, overwriting 3 previously correct outputs (dropping from 20 to 17 tasks). Consequently, refinement is non-monotonic, and the number of refinement iterations should be determined via adaptive validation criteria rather than applied as a fixed inference routine.

Analysis of Baseline Reproduction Gap

While Fletcher-Hill (Fletcher-Hill 2024) reports a TTT + Refined score of 20/114 (17.54%) for Mini-ARC-v12, our independently trained baseline achieves 5/114 (4.39%). This disparity stems primarily from pre-training dataset scale and composition. Whereas the original Mini-ARC model was pre-trained on 830,648 examples drawn from three diverse sources (RE-ARC, BARC Heavy, and ARC-HTML), our local pre-training corpus was restricted to 186,556 examples across 391 generator families from a reduced 12×12 RE-ARC subset—approximately 4.5 times smaller and lacking the structural variety of BARC and HTML puzzles.

Importantly, this baseline reproduction gap does not diminish the internal validity of our augmentation intervention: all three evaluated configurations share the exact same checkpoint, seed, evaluation split, and TTT hyperparameters.

Theoretical Limit Analysis

Our empirical findings highlight fundamental properties regarding context diversity and test-time adaptation limits on ARC-AGI:

- **Symmetry-Aware Sampling vs. Naive Voting:** Expanding TTT adaptation provides meaningful variation during optimization and potentially allows the model to overcome single-view spatial biases.
- **Context Density Bottleneck:** Context expansion via extra compatible pairs further improves models results, showing that target quantity directly dictates the quality of test-time gradient updates in compact vision models.

Conclusion

In this work, we conducted a rigorous, paired empirical evaluation of test-time training (TTT) adaptation, multi-view data augmentation, and dense demonstration expansion on an independently trained `mini-arc-v12` vision Transformer across 114 ARC-AGI tasks.

Our findings demonstrate that strong data augmentation—sampling up to 256 identity-anchored items across D_4 geometric symmetries, color permutations, and demonstration orderings, followed by canonical hierarchical consensus voting—more than doubles exact-match task resolution from 4.39% to 9.65% (11/114 tasks) and increases cell accuracy from 89.80% to 91.40%. Furthermore, context expansion via privileged verified demonstrations (ARC-GEN) achieves 14.91% (17/114 tasks) post-refinement and 17.54% (20/114 tasks) pre-refinement, demonstrating that demonstration sample size remains a primary bottleneck during test-time gradient adaptation. Crucially, our experiment reveals that iterative refinement passes are non-monotonic, boosting augmented predictions (+2 tasks) but degrading performance under dense supervision (-3 tasks). These results establish that symmetry invariance reliably amplifies few-shot reasoning, while iterative self-correction requires adaptive control to avoid destructive drift.

Future Work

Future work includes: (1) integrating confidence-based or demonstration-validated early stopping for test-time refinement passes to prevent regression in high-density contexts; (2) scaling pre-training corpora with multi-source synthetic generators (such as BARC Heavy and ARC-HTML) to assess

whether data augmentation gains amplify on higher-capacity base checkpoints; (3) incorporating lightweight program synthesis or search mechanisms to overcome the architectural saturation limits of pure vision Transformers; and (4) exploring adaptive patch tokenization to bypass fixed 12×12 grid constraints on the full ARC-AGI benchmark.

References

Akyürek, E.; Damani, M.; Zweiger, A.; Qiu, L.; Guo, H.; Pari, J.; Kim, Y.; and Andreas, J. 2025. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. arXiv:2411.07279.

Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.

Fletcher-Hill, P. 2024. Mini-ARC: Solving Abstraction and Reasoning Puzzles with Small Transformer Models. GitHub Repository and Report.

Franzen, D.; Disselhoff, J.; and Hartmann, D. 2024. The LLM ARChitect: Solving ARC-AGI Is A Matter of Perspective. arXiv:2505.07859.

Hodel, M. 2024. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353.

Li, W.-D.; Hu, K.; Larsen, C.; Wu, Y.; Alford, S.; Woo, C.; Dunn, S. M.; Tang, H.; Naim, M.; Nguyen, D.; Zheng, W.-L.; Tavares, Z.; Pu, Y.; and Ellis, K. 2024. Combining Induction and Transduction for Abstract Reasoning. arXiv:2411.02272.

Moffitt, M. D. 2025. ARC-GEN: A Mimetic Procedural Benchmark Generator for the Abstraction and Reasoning Corpus. arXiv:2511.00162.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2023. Attention Is All You Need. arXiv:1706.03762.